

M2_Regressió_Lineal_Múltiple_best_features

December 12, 2025

```
[21]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_diabetes
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

Carreguem les dades:

```
[22]: diabetes = load_diabetes()
df = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)
df['target'] = diabetes.target
```

Recordem els valors obtinguts amb el model de regressió lineal simple (un sol “feature”):

```
[23]: X_simple = df[['bmi']]
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(X_simple, y, test_size=0.2,
    random_state=42)

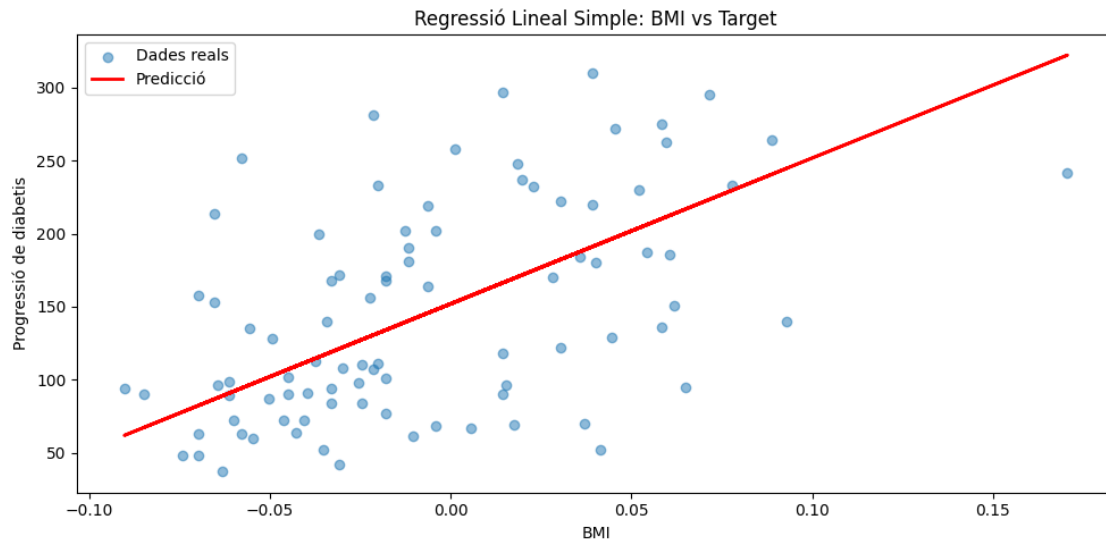
model_simple = LinearRegression()
model_simple.fit(X_train, y_train)
y_pred_simple = model_simple.predict(X_test)

print(f"\nRegressió Simple (només BMI):")
print(f"  R² Train: {model_simple.score(X_train, y_train):.4f}")
print(f"  R² Test:  {r2_score(y_test, y_pred_simple):.4f}")
print(f"  MSE: {mean_squared_error(y_test, y_pred_simple):.2f}")
print(f"  MAE: {mean_absolute_error(y_test, y_pred_simple):.2f}")
```

Regressió Simple (només BMI):

R² Train: 0.3657
R² Test: 0.2334
MSE: 4061.83
MAE: 52.26

```
[24]: plt.figure(figsize=(10, 5))
plt.scatter(X_test, y_test, alpha=0.5, label='Dades reals')
plt.plot(X_test, y_pred_simple, color='red', linewidth=2, label='Predicció')
plt.xlabel('BMI')
plt.ylabel('Progressió de diabetis')
plt.title('Regressió Lineal Simple: BMI vs Target')
plt.legend()
plt.tight_layout()
plt.show()
```



Afegir més variables en aporta més informació per entrenar el model?

Intentarem resoldre aquesta pregunta.

0.0.1 Regressió lineal múltiple (amb totes les variables disponibles)

La regressió lineal múltiple estén el model simple a múltiples variables:

$$\hat{y} = \text{intercept} + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

On:

* $\hat{y}$ = predicció (target) * intercept (valor base quan totes les x són 0) * $\beta_1, \beta_2, \dots, \beta_n$ = coeficients (pes de cada feature) * $x_1, x_2, \dots, x_n$ = features (variables independents)

INTERPRETACIÓ dels coeficients : Mesuren l'impacte de cada feature: * $\beta_i > 0$: quan x_i augmenta 1 unitat, $\hat{y}$ augmenta β_i unitats * $\beta_i < 0$: quan x_i augmenta 1 unitat, $\hat{y}$ disminueix $|\beta_i|$ unitats * $|\beta_i|$ gran: la feature té més influència en la predicció

```
[25]: X_train_all, X_test_all, y_train_all, y_test_all = train_test_split(df.
↳ drop(columns='target'), df['target'], test_size=0.2, random_state=42)
```

```
[26]: model_all = LinearRegression()
model_all.fit(X_train_all, y_train_all)
y_pred_all = model_all.predict(X_test_all)
```

```
[27]: print(f"Model amb TOTES les variables:")
print(f"  R² Train: {model_all.score(X_train_all, y_train_all):.4f}")
print(f"  R² Test:  {r2_score(y_test_all, y_pred_all):.4f}")
print(f"  MSE Test: {mean_squared_error(y_test_all, y_pred_all):.2f}")
print(f"  MAE Test: {mean_absolute_error(y_test_all, y_pred_all):.2f}")
```

Model amb TOTES les variables:

```
R² Train: 0.5279
R² Test:  0.4526
MSE Test: 2900.19
MAE Test: 42.79
```

Què ens diuen aquests resultats? Mirem la diferència entre TRAIN i TEST per veure possible overfitting.

```
[28]: r2_train = model_all.score(X_train_all, y_train_all)
r2_test = r2_score(y_test_all, y_pred_all)

diff_r2 = r2_train - r2_test
```

```
[29]: print(f"  R² Train - R² Test = {diff_r2:.4f}")
if diff_r2 < 0.02:
    print(f"Diferència petita (Model generalitza bé)")
elif diff_r2 < 0.05:
    print(f"Diferència acceptable (Model generalitza adequadament)")
elif diff_r2 < 0.10:
    print(f"Diferència moderada (Possible lleu overfitting)")
else:
    print(f"Diferència gran (Overfitting! Model memoritza les dades de train)")
```

```
R² Train - R² Test = 0.0753
```

Diferència moderada (Possible lleu overfitting)

EN RESUM: * Train > Test: model s'ajusta massa a dades d'entrenament * Test similar a Train: model generalitza bé a noves dades

```
[30]: # Mostrar importància de cada feature (coeficients)
feature_importance = pd.DataFrame({
    'Feature': df.columns[:-1],
    'Coefficient': model_all.coef_,
    'Coefficient (abs)': abs(model_all.coef_)
})

feature_importance.sort_values('Coefficient (abs)', ascending=False)
```

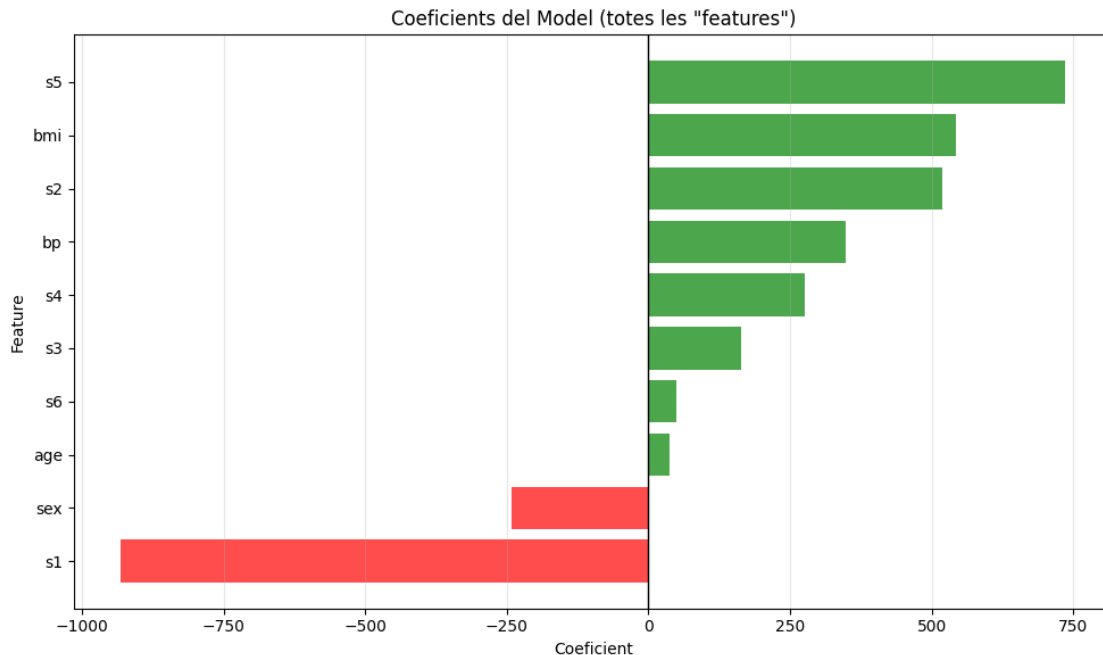
```
[30]:
```

	Feature	Coefficient	Coefficient (abs)
4	s1	-931.488846	931.488846
8	s5	736.198859	736.198859
2	bmi	542.428759	542.428759
5	s2	518.062277	518.062277
3	bp	347.703844	347.703844
7	s4	275.317902	275.317902
1	sex	-241.964362	241.964362
6	s3	163.419983	163.419983
9	s6	48.670657	48.670657
0	age	37.904021	37.904021

Com interpretar aquests coeficients? * Valor positiu: quan la *feature* augmenta, target augmenta.
 * Valor negatiu: quan la *feature* augmenta, target disminueix * Valor absolut gran: més impacte en la predicció.

Important: els coeficients depenen de l'escala de les “features”. Per comparar correctament el pes dels *features*, caldria estandarditzar/normalitzar les dades.

```
[31]: plt.figure(figsize=(10, 6))
sorted_features = feature_importance.sort_values('Coefficient', ascending=True)
colors = ['red' if x < 0 else 'green' for x in sorted_features['Coefficient']]
plt.barh(sorted_features['Feature'], sorted_features['Coefficient'],
         color=colors, alpha=0.7)
plt.xlabel('Coefficient')
plt.ylabel('Feature')
plt.title('Coeficients del Model (totes les "features")')
plt.axvline(x=0, color='black', linestyle='-', linewidth=1)
plt.grid(True, alpha=0.3, axis='x')
plt.tight_layout()
plt.show()
```



Algunes preguntes que ens hauriem de fer: * Hi ha *features* redundants o poc útils (amb poc pes)? * Podem millorar el model seleccionant només algunes *features*? * Hi ha relacions no lineals que no estem capturant?

Com podem saber quins són les “millors” *features*? (més influents)

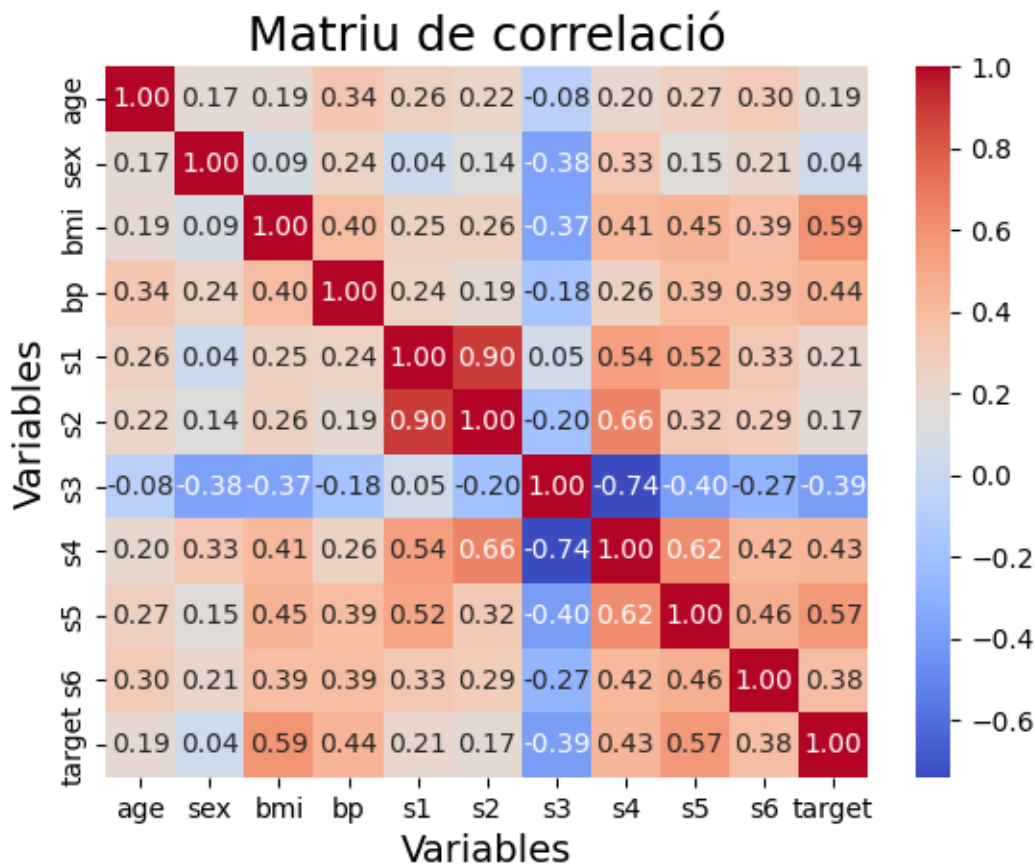
A través de la MATRIU DE CORRELACIÓ:

La correlació mesura la relació entre dues variables: * +1: correlació positiva (si una puja, l'altra també) * 0: no hi ha correlació * -1: correlació negativa (si una puja, l'altra baixa)

Quines característiques han de tenir bones *features*: * Features amb ALTA correlació amb el target (de valor absolut entre 0.3-0.5). * Però evitem features molt correlacionades entre elles (multicolinearitat). Si dues features estan molt correlacionades, n'hi ha prou amb una.

```
[32]: import seaborn as sns
corr_matrix=df.corr()

sns.heatmap(corr_matrix,annot=True,fmt='.2f',cmap='coolwarm')
plt.title('Matriu de correlació',fontsize=18)
plt.xlabel('Variables',fontsize=14)
plt.ylabel('Variables',fontsize=14)
plt.show()
```



INTERPRETACIÓ: * Valors propers a +1: correlació positiva forta * Valors propers a -1: correlació negativa forta * Valors propers a 0: poca o cap correlació

```
[33]: # Correlació de cada variable amb el target
target_correlation = corr_matrix['target'].sort_values(ascending=False)
target_correlation
```

```
[33]: target      1.000000
bmi          0.586450
s5           0.565883
bp           0.441482
s4           0.430453
s6           0.382483
s1           0.212022
age          0.187889
s2           0.174054
sex          0.043062
s3          -0.394789
Name: target, dtype: float64
```

```
[34]: threshold = 0.4
best_features = target_correlation[
    (abs(target_correlation) >= threshold) & (target_correlation.index != 'target')
].index.tolist()

best_features
```

```
[34]: ['bmi', 's5', 'bp', 's4']
```

També comprovem la multicolinearietat entre les features seleccionades:

```
[35]: feature_corr_matrix = df[best_features].corr()

high_corr_pairs = []
for i in range(len(best_features)):
    for j in range(i+1, len(best_features)):
        corr_value = feature_corr_matrix.iloc[i, j]
        if abs(corr_value) > 0.7: # Threshold per multicolinearietat
            high_corr_pairs.append((best_features[i], best_features[j], corr_value))

for feat1, feat2, corr in high_corr_pairs:
    print(f"{feat1} - {feat2}: {corr:.4f}")
```

```
[36]: # best_features = ['bmi', 's5', 'bp'] # Amb un threshold de 0.6 encara podem
      # ajustar una mica més el model
```

Finalment fem l'entrenament amb els 'best' features:

```
[37]: X_selected = df[best_features].values
X_train_sel, X_test_sel, y_train_sel, y_test_sel = train_test_split(
    X_selected, y, test_size=0.2, random_state=42
)

model_selected = LinearRegression()
model_selected.fit(X_train_sel, y_train_sel)
y_pred_selected = model_selected.predict(X_test_sel)

[38]: print(f"Model amb variables seleccionades (best features):")
print(f"  R² Train: {model_selected.score(X_train_sel, y_train_sel):.4f}")
print(f"  R² Test:  {r2_score(y_test_sel, y_pred_selected):.4f}")
print(f"  MSE Test: {mean_squared_error(y_test_sel, y_pred_selected):.2f}")
print(f"  MAE Test: {mean_absolute_error(y_test_sel, y_pred_selected):.2f}")
```

Model amb variables seleccionades (best features):

```
  R² Train: 0.4850
  R² Test:  0.4526
  MSE Test: 2900.29
```

MAE Test: 44.31

```
[39]: r2_train = model_selected.score(X_train_sel, y_train_sel)
      r2_test = r2_score(y_test_sel, y_pred_selected)

      diff_r2 = r2_train - r2_test
```

```
[40]: print(f"    R² Train - R² Test = {diff_r2:.4f}")
      if diff_r2 < 0.02:
          print(f"Diferència petita (Model generalitza bé)")
      elif diff_r2 < 0.05:
          print(f"Diferència acceptable (Model generalitza adequadament)")
      elif diff_r2 < 0.10:
          print(f"Diferència moderada (Possible lleu overfitting)")
      else:
          print(f"Diferència gran (Overfitting! Model memoritza les dades de train)")
```

R² Train - R² Test = 0.0324

Diferència acceptable (Model generalitza adequadament)

És important: * Analitzar correlacions ABANS d'entrenar és molt important * Usar variables amb alta correlació amb el 'target', ja que millora el model * Evitar variables amb correlació baixa (<0.3), ja que redueix "soroll" * Vigilar multicolinealitat (variables correlacionades entre elles

Més variables NO sempre = millor model

Així doncs els passos a seguir per entrenar un model són:

1. Carregar les dades
2. Fer un anàlisi exploratori de les dades (EDA)
3. Preparar/Triar la variable/variables d'entrada i sortida del model
4. Dividir les dades entre TRAIN/TEST
5. Instanciar el model (crear el model del tipus necessari)
6. Entrenar el model
7. Fer una predicció
8. Avaluar el resultat (ajudat per mètriques).